

Series Temporales

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezf@udesa.edu.ar y lo revisamos.

1. Introducción a Series Temporales

Una **serie temporal** (o serie de tiempo) es una secuencia de observaciones de una variable medidas a lo largo del tiempo en intervalos regulares. A diferencia de los datos que venimos viendo en clases anteriores (donde cada observación es independiente), en las series temporales el orden de las observaciones importa y existe una dependencia temporal entre ellas.

1.1. Ejemplos de Series Temporales

Las series temporales aparecen en múltiples dominios:

- **Finanzas:** Precio de acciones, tipo de cambio, índices bursátiles.
- **Economía:** PBI trimestral, tasa de desempleo, inflación mensual.
- **Retail:** Ventas diarias de productos, demanda de servicios
- **Clima:** Temperatura diaria, precipitaciones mensuales.
- **Tecnología:** Tráfico web, uso de CPU, número de usuarios activos.

1.2. Características Principales

Las series temporales tienen tres características fundamentales que las distinguen de otros tipos de datos:

1. **Dependencia temporal:** El valor en el tiempo t puede depender de valores en tiempos anteriores $t - 1, t - 2, \dots$
2. **Autocorrelación:** La correlación de una variable consigo misma en diferentes momentos del tiempo. Por ejemplo, si hace calor hoy, es probable que mañana también haga calor.

3. **Orden:** El orden de las observaciones no puede alterarse sin perder información. No podemos mezclar las observaciones como haríamos con un dataset tradicional.

1.3. Análisis Descriptivo vs Predictivo

Cuando trabajamos con series temporales, los objetivos principales suelen encuadrarse en dos enfoques: el análisis descriptivo y el análisis predictivo. El análisis descriptivo se orienta a entender los patrones históricos y tendencias de la serie, identificar estacionalidades y ciclos, detectar anomalías o valores atípicos y descomponer la serie en sus diferentes componentes. Por su parte, el análisis predictivo (o forecasting) tiene como propósito predecir valores futuros de la serie, estimar la demanda futura para fines de planificación, proyectar tendencias tanto a corto como a largo plazo, y generar intervalos de confianza para las predicciones.

Ambos enfoques son complementarios: comprender la estructura de la serie mediante el análisis descriptivo resulta fundamental para construir mejores modelos de predicción en el análisis predictivo.

2. Componentes de una Serie Temporal

Una serie temporal Y_t se puede descomponer en cuatro componentes principales:

- **Tendencia (T_t):** Es el comportamiento de largo plazo de la serie. Representa el crecimiento o decrecimiento sostenido a lo largo del tiempo. Por ejemplo, el aumento poblacional o el crecimiento de ventas de una empresa.
- **Ciclo (C_t):** Son fluctuaciones de mediano plazo que no tienen una frecuencia fija. En economía, los ciclos de expansión y recesión suelen durar varios años. A menudo se combina con la tendencia en un componente $T_t \cdot C_t$.
- **Estacionalidad (S_t):** Son patrones que se repiten con periodicidad fija (mensual, trimestral, anual). Por ejemplo, mayores ventas en diciembre, más consumo eléctrico en verano/invierno, o picos de tráfico web los fines de semana.

- **Ruido (ϵ_t):** Es la componente aleatoria o irregular que no puede explicarse por las otras componentes. Representa variaciones impredecibles debidas a eventos aleatorios.

3. Media Móvil

La **media móvil** (moving average) es uno de los métodos más simples para suavizar una serie temporal y hacer predicciones. La idea es promediar las últimas k observaciones para eliminar el ruido y capturar la tendencia.

3.1. Media Móvil Simple (SMA)

La media móvil simple de orden k en el tiempo t se calcula como:

$$\text{SMA}_k(t) = \frac{1}{k} \sum_{i=0}^{k-1} Y_{t-i} = \frac{Y_t + Y_{t-1} + \dots + Y_{t-k+1}}{k}$$

Para predecir el valor en $t + 1$, usamos:

$$\hat{Y}_{t+1} = \text{SMA}_k(t)$$

Importante: Para predecir más de un paso adelante (por ejemplo, $t+2$, $t+3$, etc.), la media móvil simple siempre usa el mismo valor: $\hat{Y}_{t+h} = \text{SMA}_k(t)$ para cualquier $h \geq 1$. Esto significa que todas las predicciones futuras serán iguales al último promedio calculado, lo cual es una limitación importante del método. Solo es útil para predicciones a muy corto plazo (1-2 pasos adelante).

3.1.1. Elección de k :

- k pequeño (3-5): Responde rápido a cambios, pero más sensible al ruido
- k grande (10-20): Suaviza más, pero responde lento a cambios reales

3.2. Ejercicio: Cálculo Manual de Media Móvil

Consideremos las siguientes ventas diarias de un producto (en unidades):

Día	1	2	3	4	5	6	7	8	9	10
Ventas	23	27	24	30	28	26	31	29	27	33

Calculemos la media móvil de orden $k = 3$:

- **Día 3:** $\text{SMA}_3(3) = \frac{23+27+24}{3} = \frac{74}{3} = 24,67$
- **Día 4:** $\text{SMA}_3(4) = \frac{27+24+30}{3} = \frac{81}{3} = 27,00$
- **Día 5:** $\text{SMA}_3(5) = \frac{24+30+28}{3} = \frac{82}{3} = 27,33$
- **Día 6:** $\text{SMA}_3(6) = \frac{30+28+26}{3} = \frac{84}{3} = 28,00$
- **Día 7:** $\text{SMA}_3(7) = \frac{28+26+31}{3} = \frac{85}{3} = 28,33$
- **Día 8:** $\text{SMA}_3(8) = \frac{26+31+29}{3} = \frac{86}{3} = 28,67$
- **Día 9:** $\text{SMA}_3(9) = \frac{31+29+27}{3} = \frac{87}{3} = 29,00$
- **Día 10:** $\text{SMA}_3(10) = \frac{29+27+33}{3} = \frac{89}{3} = 29,67$

Predicción para el día 11: $\hat{Y}_{11} = \text{SMA}_3(10) = 29,67 \approx 30$ unidades

Predicciones para días futuros: Como no tenemos datos reales del día 11 en adelante, todas las predicciones futuras usan el mismo valor:

- Día 11: $\hat{Y}_{11} = \text{SMA}_3(10) = 29,67$
- Día 12: $\hat{Y}_{12} = \text{SMA}_3(10) = 29,67$ (mismo valor)
- Día 13: $\hat{Y}_{13} = \text{SMA}_3(10) = 29,67$ (mismo valor)
- Día 14: $\hat{Y}_{14} = \text{SMA}_3(10) = 29,67$ (mismo valor)

Por eso la media móvil simple solo es útil para predicciones a muy corto plazo. Para predicciones más lejanas, necesitamos métodos más sofisticados que puedan modelar tendencias y estacionalidad.

3.3. Media Móvil Ponderada (WMA)

En algunos casos queremos dar más peso a las observaciones recientes:

$$\text{WMA}_k(t) = \sum_{i=0}^{k-1} w_i \cdot Y_{t-i}$$

donde w_i son los pesos (con $\sum w_i = 1$) y típicamente $w_0 > w_1 > \dots > w_{k-1}$.

Por ejemplo, para $k = 3$ podríamos usar $w_0 = 0,5, w_1 = 0,3, w_2 = 0,2$:

$$\text{WMA}_3(10) = 0,5 \times 33 + 0,3 \times 27 + 0,2 \times 29 = 16,5 + 8,1 + 5,8 = 30,4$$

3.4. Implementación en Python

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Datos del ejemplo
6 ventas = [23, 27, 24, 30, 28, 26, 31, 29, 27, 33]
7 dias = list(range(1, 11))
8
9 # Crear serie temporal
10 ts = pd.Series(ventas, index=dias)
11
12 # Media movil simple de orden 3
13 sma_3 = ts.rolling(window=3).mean()
14
15 # Media movil simple de orden 5
16 sma_5 = ts.rolling(window=5).mean()
17
18 # Prediccion para el proximo periodo
19 prediccion = sma_3.iloc[-1]
20 print(f"Prediccion para dia 11: {prediccion:.2f}")
21
22 # Visualizacion
23 plt.figure(figsize=(10, 5))
24 plt.plot(dias, ventas, 'o-', label='Ventas Reales', linewidth=2)
25 plt.plot(dias, sma_3, 's--', label='SMA(3)', linewidth=2)
26 plt.plot(dias, sma_5, '^--', label='SMA(5)', linewidth=2)
27 plt.xlabel('Dia')
28 plt.ylabel('Ventas')
29 plt.title('Ventas con Medias Moviles')
30 plt.legend()
31 plt.grid(True, alpha=0.3)
32 plt.show()
```

3.5. Modelos de Descomposición

Los componentes principales de una serie temporal dijimos que son:

- **Tendencia** (T_t): el comportamiento a largo plazo o dirección general de la serie.
- **Estacionalidad** (S_t): patrones que se repiten de forma regular en períodos específicos.
- **Ruido o componente aleatorio** (ϵ_t): las fluctuaciones impredecibles o residuales.

Existen dos formas principales de combinar estos componentes:

Modelo Aditivo:

$$Y_t = T_t + S_t + \epsilon_t$$

Este modelo es apropiado cuando la magnitud de las variaciones estacionales es constante a lo largo del tiempo. Por ejemplo, si las ventas aumentan siempre en 1000 unidades en diciembre, independientemente del nivel base de ventas.

Modelo Multiplicativo:

$$Y_t = T_t \times S_t \times \epsilon_t$$

Este modelo es apropiado cuando las variaciones estacionales son proporcionales al nivel de la serie. Por ejemplo, si las ventas aumentan un 20 % en diciembre, la cantidad absoluta dependerá del nivel de ventas base.

3.6. Ejercicio: Descomposición Manual de una Serie

Consideremos las siguientes ventas trimestrales (en miles de unidades) durante 3 años:

Año	Q1	Q2	Q3	Q4
2021	10	12	11	15
2022	11	13	12	16
2023	12	14	13	17

Vamos a descomponer esta serie usando un modelo aditivo.

3.6.1. Paso 1: Calcular la Tendencia

Usamos una media móvil centrada de 4 períodos (1 año completo). Para eliminar la estacionalidad anual, usamos una ventana que se desliza de 4 trimestres consecutivos. Como el orden es par, debemos elegir cómo balancear la ventana: podemos usar 2 trimestres anteriores + actual + 1 siguiente, o 1 anterior + actual + 2 siguientes. Elegimos la segunda opción (1 anterior + actual + 2 siguientes) porque así podemos empezar a calcular desde Q2 del primer año, sin necesidad de datos del año anterior. Ambas opciones cubren un año completo y eliminan la estacionalidad:

Trimestre	Ventas	Cálculo	Tendencia T_t
2021-Q1	10	-	-
2021-Q2	12	$\frac{10+12+11+15}{4}$	12.0
2021-Q3	11	$\frac{12+11+15+11}{4}$	12.25
2021-Q4	15	$\frac{11+15+11+13}{4}$	12.25
2022-Q1	11	$\frac{15+11+13+12}{4}$	12.5
2022-Q2	13	$\frac{11+13+12+16}{4}$	13.0
2022-Q3	12	$\frac{13+12+16+12}{4}$	13.25
2022-Q4	16	$\frac{12+16+12+14}{4}$	13.5
2023-Q1	12	$\frac{16+12+14+13}{4}$	13.75
2023-Q2	14	$\frac{12+14+13+17}{4}$	14.0
2023-Q3	13	-	-
2023-Q4	17	-	-

3.6.2. Paso 2: Calcular la Componente Estacional + Ruido

Restamos la tendencia de los valores observados: $Y_t - T_t$

Trimestre	Ventas	Tendencia	$Y_t - T_t$
2021-Q2	12	12.0	0.0
2021-Q3	11	12.25	-1.25
2021-Q4	15	12.25	2.75
2022-Q1	11	12.5	-1.5
2022-Q2	13	13.0	0.0
2022-Q3	12	13.25	-1.25
2022-Q4	16	13.5	2.5
2023-Q1	12	13.75	-1.75
2023-Q2	14	14.0	0.0

3.6.3. Paso 3: Calcular Índices Estacionales

Promediamos las diferencias para cada trimestre:

Trimestre	Índice Estacional S_t
Q1	$\frac{-1,5+(-1,75)}{2} = -1,625$
Q2	$\frac{0,0+0,0+0,0}{3} = 0,0$
Q3	$\frac{-1,25+(-1,25)}{2} = -1,25$
Q4	$\frac{2,75+2,5}{2} = 2,625$

3.6.4. Interpretación:

- La tendencia muestra un crecimiento sostenido de aproximadamente 0.25 mil unidades por trimestre
- Q1 y Q3 tienen ventas por debajo de la tendencia (temporadas bajas)
- Q4 es la temporada alta con aproximadamente 2.6 mil unidades adicionales
- Q2 se mantiene cerca de la tendencia

4. Holt-Winters

El método de Holt-Winters (también llamado **suavizamiento exponencial triple**) es una extensión del suavizamiento exponencial que permite modelar series con tendencia y estacionalidad simultáneamente. Vamos a construirlo paso a paso.

4.1. Suavizamiento Exponencial Simple

El suavizamiento exponencial simple es útil para series sin tendencia ni estacionalidad. La idea es dar más peso a las observaciones recientes:

$$\hat{Y}_{t+1} = \alpha Y_t + (1 - \alpha) \hat{Y}_t$$

donde $\alpha \in [0, 1]$ es el **parámetro de suavizamiento**:

- α cercano a 1: más peso a observaciones recientes (reacciona rápido)

- α cercano a 0: más peso a valores históricos (suaviza más)

Esta fórmula se puede reescribir como:

$$\hat{Y}_{t+1} = \hat{Y}_t + \alpha(Y_t - \hat{Y}_t)$$

Es decir, ajustamos la predicción anterior según el error cometido.

4.2. Método de Holt (Tendencia Lineal)

Holt extendió el método anterior para series con tendencia, proponiendo una versión doblemente suavizada del modelo. Esta versión agrega una ecuación adicional para modelar explícitamente la tendencia en la serie temporal. En el método de Holt, se tienen dos componentes principales que se actualizan recursivamente en cada periodo usando parámetros de suavizamiento independientes:

- **Nivel** (L_t): representa el valor base estimado de la serie en el tiempo t .
- **Tendencia** (T_t): estima el cambio (pendiente) del nivel a través del tiempo.

Ecuación del nivel:

$$L_t = \alpha Y_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

Ecuación de la tendencia:

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Predicción:

$$\hat{Y}_{t+h} = L_t + h \cdot T_t$$

donde:

- L_t es el nivel (componente base) en el tiempo t
- T_t es la tendencia en el tiempo t
- α controla el suavizamiento del nivel
- β controla el suavizamiento de la tendencia
- h es el horizonte de predicción (cuántos pasos adelante)

4.3. Holt-Winters (Tendencia + Estacionalidad)

Pasa que después vino el amigo Winters y quiso sumarle un poco más de data y agrego una tercera ecuación para la estacionalidad. Recordemos primero que el nivel (L_t) es el valor que representa el promedio de la serie en el tiempo t descontando la tendencia y la estacionalidad. Existen dos variantes dependiendo de si la estacionalidad es constante o si crece con el nivel:

Modelo Aditivo (cuando la estacionalidad es constante):

$$\begin{aligned}L_t &= \alpha(Y_t - S_{t-s}) + (1 - \alpha)(L_{t-1} + T_{t-1}) \\T_t &= \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \\S_t &= \gamma(Y_t - L_t) + (1 - \gamma)S_{t-s} \\\hat{Y}_{t+h} &= L_t + h \cdot T_t + S_{t+h-s}\end{aligned}$$

Modelo Multiplicativo (cuando la estacionalidad crece con el nivel):

$$\begin{aligned}L_t &= \alpha \frac{Y_t}{S_{t-s}} + (1 - \alpha)(L_{t-1} + T_{t-1}) \\T_t &= \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \\S_t &= \gamma \frac{Y_t}{L_t} + (1 - \gamma)S_{t-s} \\\hat{Y}_{t+h} &= (L_t + h \cdot T_t) \times S_{t+h-s}\end{aligned}$$

donde:

- s es la longitud del período estacional (ej: 12 para datos mensuales con estacionalidad anual)
- $\gamma \in [0, 1]$ controla el suavizamiento de la estacionalidad
- S_t es el índice estacional en el tiempo t

4.4. Interpretación de los Parámetros

- α (**nivel**): Qué tan rápido se adapta al valor actual. Valores altos (0.7-0.9) hacen que el modelo reaccione rápido a cambios en el nivel.

- β (**tendencia**): Qué tan rápido se adapta a cambios en la tendencia. Valores bajos (0.1-0.3) evitan que la tendencia fluctúe mucho.
- γ (**estacionalidad**): Qué tan rápido se adapta a cambios en los patrones estacionales. Valores bajos (0.1-0.2) mantienen patrones estacionales estables.

4.5. Implementación en Python

```

1 from statsmodels.tsa.holtwinters import ExponentialSmoothing
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Datos mensuales con estacionalidad (24 meses)
7 ventas = [112, 118, 132, 129, 121, 135, 148, 148, 136, 119,
8           104, 118, 115, 126, 141, 135, 125, 149, 170, 170,
9           158, 133, 114, 140]
10 fechas = pd.date_range('2022-01', periods=24, freq='MS')
11 ts = pd.Series(ventas, index=fechas)
12
13 # Ajustar modelo Holt-Winters
14 # seasonal_periods=12 indica estacionalidad anual
15 # trend='add': tendencia aditiva
16 # seasonal='add': estacionalidad aditiva
17 modelo = ExponentialSmoothing(
18     ts,
19     seasonal_periods=12,
20     trend='add',
21     seasonal='add'
22 )
23
24 # Entrenar el modelo
25 fit = modelo.fit()
26
27 # Hacer predicciones para los proximos 6 meses
28 predicciones = fit.forecast(steps=6)
29
30 print("Parametros estimados:")
31 print(f"Alpha (nivel): {fit.params['smoothing_level']:.3f}")
32 print(f"Beta (tendencia): {fit.params['smoothing_trend']:.3f}")
33 print(f"Gamma (estacionalidad): {fit.params['smoothing_seasonal']:.3f}")

```

```

34
35 print("\nPredicciones para los proximos 6 meses:")
36 print(predicciones)
37
38 # Visualizacion
39 plt.figure(figsize=(12, 6))
40 plt.plot(ts, label='Datos Historicos', marker='o')
41 plt.plot(fit.fittedvalues, label='Ajuste del Modelo',
42         linestyle='--', linewidth=2)
43 plt.plot(predicciones, label='Predicciones',
44         marker='s', linestyle='--', linewidth=2)
45 plt.xlabel('Fecha')
46 plt.ylabel('Ventas')
47 plt.title('Modelo Holt-Winters')
48 plt.legend()
49 plt.grid(True, alpha=0.3)
50 plt.show()

```

4.6. Cuándo usar Holt-Winters

El método de Holt-Winters es especialmente útil para series temporales que presentan tendencia y estacionalidad bien definidas, como datos mensuales, trimestrales o semanales con patrones repetitivos. Es comúnmente empleado en contextos donde se requieren pronósticos a corto o mediano plazo (hasta uno o dos ciclos estacionales completos), por ejemplo en sectores como retail, turismo o energía, en los que los patrones estacionales son muy marcados.

4.7. Limitaciones

Holt-Winters tiene limitaciones importantes: supone que los patrones observados en el pasado se mantendrán en el futuro y no considera el impacto de variables externas (como promociones, cambios económicos u otros factores). Además, la incertidumbre de las predicciones tiende a incrementarse rápidamente a medida que se pronostican períodos más alejados.

5. ARIMA

ARIMA (AutoRegressive Integrated Moving Average) es una familia de modelos estadísticos muy utilizada para análisis y predicción de series tempo-

rales. Es más sofisticado que los métodos anteriores y puede capturar una mayor variedad de patrones.

5.1. Estacionaridad

Antes de usar ARIMA, necesitamos entender el concepto de **estacionariedad**. Una serie es estacionaria cuando sus propiedades estadísticas (media, varianza) no cambian con el tiempo.

Señales de no estacionaridad:

- Tendencia clara (creciente o decreciente)
- Varianza que cambia con el tiempo
- Estacionalidad fuerte

Para hacer estacionaria una serie, usamos **diferenciación**: restamos cada observación de la anterior.

Primera diferencia:

$$\Delta Y_t = Y_t - Y_{t-1}$$

Segunda diferencia:

$$\Delta^2 Y_t = \Delta Y_t - \Delta Y_{t-1} = (Y_t - Y_{t-1}) - (Y_{t-1} - Y_{t-2})$$

Donde $\Delta^2 Y_t$ es la diferencia de las diferencias, es decir, aplicamos el operador diferencia dos veces consecutivas.

¿Cuántas diferencias usar? Usamos primera diferencia ($d = 1$) cuando la serie tiene tendencia lineal. Usamos segunda diferencia ($d = 2$) cuando después de la primera diferencia todavía hay tendencia (tendencia no lineal o acelerada). En la práctica, raramente se necesitan más de dos diferencias, ya que aplicar demasiadas puede eliminar información útil e introducir ruido artificial. Para determinar d , podemos visualizar la serie diferenciada (debe parecer estacionaria, sin tendencia clara) o usar pruebas estadísticas como ADF o KPSS.

5.2. Componentes de ARIMA

ARIMA tiene tres componentes, denotados como ARIMA(p, d, q):

5.2.1. AR(p) - AutoRegresivo

El componente autoregresivo predice el valor actual como una combinación lineal de p valores pasados:

$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + \epsilon_t$$

donde $\phi_1, \dots, \phi_p$ son los coeficientes y ϵ_t es ruido blanco.

¿De dónde salen los coeficientes y el ruido blanco? Cuando ajustamos un modelo ARIMA a nuestros datos, el software (por ejemplo, Python con statsmodels) **calcula automáticamente** los valores de los coeficientes $\phi_1, \dots, \phi_p$ que mejor explican los datos históricos. Es similar a una regresión lineal: los coeficientes se encuentran minimizando el error entre los valores reales y los predichos. El ruido blanco ϵ_t se agrega a la ecuación para que “dé”, es decir, para que el lado izquierdo (Y_t) sea igual al lado derecho (la combinación lineal de valores pasados). Sin el ruido blanco, la ecuación no se cumpliría exactamente porque siempre hay una diferencia entre el valor real y lo que predice el modelo usando solo los valores pasados. Esta diferencia es aleatoria e impredecible, por eso se llama “ruido”.

Intuición: El valor de hoy depende de los valores de los últimos p días. Por ejemplo, si AR(1) con $\phi_1 = 0,7$, entonces el 70 % del valor de hoy se explica por el valor de ayer.

5.2.2. I(d) - Integrado

El componente de integración indica cuántas veces diferenciamos la serie para hacerla estacionaria:

- $d = 0$: La serie ya es estacionaria
- $d = 1$: Aplicamos primera diferencia
- $d = 2$: Aplicamos segunda diferencia (raro en la práctica)

5.2.3. MA(q) - Media Móvil

El componente de media móvil modela el valor actual como una combinación lineal de q errores pasados:

$$Y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

donde $\theta_1, \dots, \theta_q$ son los coeficientes y ϵ_t son los errores (ruido blanco, igual que en AR).

¿De dónde salen los coeficientes? Al igual que en AR, los coeficientes $\theta_1, \dots, \theta_q$ se calculan automáticamente cuando ajustamos el modelo, encontrando los valores que mejor explican los datos históricos.

Intuición: El valor de hoy depende de shocks o errores aleatorios de los últimos q períodos.

5.3. Modelo ARIMA(p,d,q) Completo

Combinando todo, un modelo ARIMA(p,d,q) se define sobre la serie diferenciada d veces:

$$\Delta^d Y_t = c + \sum_{i=1}^p \phi_i \Delta^d Y_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \epsilon_t$$

5.4. Selección de Parámetros

Elegir los valores correctos de p , d y q es crucial para tener un buen modelo ARIMA. Existen varias estrategias:

5.4.1. Determinar d (diferenciación)

Como vimos antes, d se determina según la estacionaridad de la serie:

- Visualizar la serie: si tiene tendencia, probablemente necesite $d = 1$ o $d = 2$
- Pruebas estadísticas: tests como ADF (Augmented Dickey-Fuller) o KPSS indican si la serie es estacionaria
- Regla práctica: empezar con $d = 1$ y verificar si la serie diferenciada parece estacionaria

5.4.2. Determinar p y q usando ACF y PACF

Los gráficos de autocorrelación (ACF) y autocorrelación parcial (PACF) ayudan a identificar p y q :

ACF (Autocorrelation Function): Muestra la correlación entre Y_t y Y_{t-k} para diferentes lags k . Si el ACF decae lentamente (muchos lags significativos), sugiere que necesitamos diferenciación o términos MA.

PACF (Partial Autocorrelation Function): Muestra la correlación directa entre Y_t y Y_{t-k} controlando por los valores intermedios. Los picos significativos en el PACF sugieren términos AR.

Reglas prácticas:

- Si el PACF tiene un “corte” después del lag p (valores no significativos después), entonces p es el número de lags significativos
- Si el ACF tiene un “corte” después del lag q , entonces q es el número de lags significativos
- Si ambos decaen gradualmente, puede ser necesario más diferenciación o un modelo más complejo

5.4.3. Criterios de Información (AIC/BIC)

Una vez que tenemos candidatos de modelos, comparamos su calidad usando criterios de información:

AIC (Akaike Information Criterion): Penaliza modelos complejos pero menos que BIC. Útil para comparar modelos con diferentes números de parámetros. Menor AIC indica mejor modelo.

BIC (Bayesian Information Criterion): Penaliza más la complejidad que AIC, favoreciendo modelos más simples. También menor es mejor.

Regla práctica: Probar varios modelos con diferentes valores de p y q , calcular AIC/BIC, y elegir el modelo con menor valor. BIC suele preferirse cuando queremos modelos más parsimoniosos (simples).

5.4.4. Auto ARIMA

Auto ARIMA (implementado en Python con `pmdarima`) prueba automáticamente diferentes combinaciones de p , d y q y selecciona el modelo con mejor AIC. Es muy útil cuando no estamos seguros de los valores, pero es importante entender qué está haciendo para interpretar los resultados correctamente.

5.5. Implementación en Python

```
1 from statsmodels.tsa.arima.model import ARIMA
2 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 # Datos de ejemplo: ventas mensuales
8 ventas = [200, 205, 210, 215, 218, 220, 225, 230, 232, 235,
9           240, 245, 242, 248, 253, 258, 260, 265, 270, 275,
10          278, 280, 285, 290]
11 fechas = pd.date_range('2022-01', periods=24, freq='MS')
12 ts = pd.Series(ventas, index=fechas)
13
14 # Visualizar ACF y PACF para diagnostico
15 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
16 plot_acf(ts, ax=axes[0], lags=10)
17 plot_pacf(ts, ax=axes[1], lags=10)
18 plt.show()
19
20 # Ajustar modelo ARIMA(1,1,1)
21 # p=1: un termino autoregresivo
22 # d=1: una diferenciacion
23 # q=1: un termino de media movil
24 modelo = ARIMA(ts, order=(1, 1, 1))
25 fit = modelo.fit()
26
27 # Resumen del modelo
28 print(fit.summary())
29
30 # Hacer predicciones para los proximos 6 meses
31 predicciones = fit.forecast(steps=6)
32 print("\nPredicciones:")
33 print(predicciones)
34
35 # Visualizacion
36 plt.figure(figsize=(12, 6))
37 plt.plot(ts, label='Datos Historicos', marker='o')
38 plt.plot(fit.fittedvalues, label='Ajuste ARIMA',
39          linestyle='--', linewidth=2)
40 plt.plot(predicciones, label='Predicciones',
41          marker='s', linestyle='--', linewidth=2)
42 plt.xlabel('Fecha')
43 plt.ylabel('Ventas')
```

```

44 plt.title('Modelo ARIMA(1,1,1)')
45 plt.legend()
46 plt.grid(True, alpha=0.3)
47 plt.show()
48
49 # Diagnosticos del modelo
50 fit.plot_diagnostics(figsize=(12, 8))
51 plt.show()

```

5.6. Auto ARIMA

Para encontrar automáticamente los mejores parámetros:

```

1 from pmdarima import auto_arima
2
3 # Buscar el mejor modelo automaticamente
4 modelo_auto = auto_arima(
5     ts,
6     start_p=0, max_p=3,
7     start_q=0, max_q=3,
8     d=None, # Se determina automaticamente
9     seasonal=False,
10    trace=True, # Muestra el proceso de busqueda
11    error_action='ignore',
12    suppress_warnings=True,
13    stepwise=True
14 )
15
16 print(modelo_auto.summary())
17
18 # Predicciones
19 predicciones_auto = modelo_auto.predict(n_periods=6)
20 print("\nPredicciones con Auto ARIMA:")
21 print(predicciones_auto)

```

6. Evaluación de Modelos

Una vez que tenemos varios modelos (Media Móvil, Holt-Winters, ARIMA), necesitamos compararlos y evaluar su desempeño.

6.1. Train-Test Split Temporal

A diferencia de otros problemas de machine learning, en series temporales **NO podemos mezclar los datos**. Debemos respetar el orden temporal:

```
1 # CORRECTO para series temporales
2 train_size = int(len(ts) * 0.8)
3 train = ts[:train_size]
4 test = ts[train_size:]
5
6 # INCORRECTO (no hacer esto)
7 # from sklearn.model_selection import train_test_split
8 # train, test = train_test_split(ts, test_size=0.2) # NO!
```

6.2. Métricas de Evaluación

6.2.1. MAE (Mean Absolute Error):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|$$

Promedio de errores en valor absoluto. Fácil de interpretar (mismas unidades que Y).

6.2.2. MSE (Mean Squared Error):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Penaliza más los errores grandes. Útil para optimización pero difícil de interpretar.

6.2.3. RMSE (Root Mean Squared Error):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$$

Similar a MAE pero penaliza errores grandes. Mismas unidades que Y .

6.2.4. MAPE (Mean Absolute Percentage Error):

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{Y_i - \hat{Y}_i}{Y_i} \right|$$

Error porcentual promedio. Útil para comparar series de diferentes escalas. Problema: indefinido si $Y_i = 0$.

6.3. Cross-Validation para Series Temporales

A diferencia de otros problemas de machine learning, en series temporales **NO podemos usar cross-validation tradicional** (mezclar los datos aleatoriamente) porque violaría la dependencia temporal. En su lugar, usamos **Time Series Cross-Validation**, que respeta el orden temporal de los datos.

6.3.1. ¿Por qué no podemos mezclar los datos?

Si mezcláramos los datos aleatoriamente, estaríamos usando información del futuro para predecir el pasado, lo cual no tiene sentido en la práctica. Además, romperíamos las dependencias temporales que son fundamentales en las series temporales.

6.3.2. Estrategias de Cross-Validation Temporal

Existen dos estrategias principales. Para entenderlas mejor, imaginemos que tenemos datos de 12 meses (enero a diciembre):

1. Rolling Window (Ventana Móvil): El tamaño del conjunto de entrenamiento se mantiene constante. En cada fold, “deslizamos” la ventana hacia adelante.

- **Fold 1:** Entrenar con meses 1-6, probar con meses 7-8
- **Fold 2:** Entrenar con meses 3-8, probar con meses 9-10
- **Fold 3:** Entrenar con meses 5-10, probar con meses 11-12

Notar que el tamaño del entrenamiento siempre es 6 meses, pero la ventana se “desliza” hacia adelante.

2. Expanding Window (Ventana Expansiva): El tamaño del conjunto de entrenamiento crece en cada fold. El inicio se mantiene fijo y solo avanzamos el final.

- **Fold 1:** Entrenar con meses 1-4, probar con meses 5-6
- **Fold 2:** Entrenar con meses 1-6, probar con meses 7-8
- **Fold 3:** Entrenar con meses 1-8, probar con meses 9-10
- **Fold 4:** Entrenar con meses 1-10, probar con meses 11-12

Notar que siempre empezamos desde el mes 1, pero el entrenamiento “se expande” incluyendo más datos históricos en cada fold.

¿Cuándo usar cada una?

- **Rolling Window:** Útil cuando queremos evaluar el desempeño del modelo con una cantidad fija de datos históricos. Simula mejor el escenario real donde tenemos una ventana de tiempo limitada (por ejemplo, siempre usamos los últimos 6 meses para predecir).
- **Expanding Window:** Útil cuando queremos aprovechar toda la información histórica disponible. Simula mejor el caso donde acumulamos datos a lo largo del tiempo y siempre usamos todo el historial disponible.

6.3.3. Implementación con TimeSeriesSplit

TimeSeriesSplit de scikit-learn implementa rolling window por defecto:

```
1 from sklearn.model_selection import TimeSeriesSplit
2 from sklearn.metrics import mean_absolute_error,
  mean_squared_error
3 import numpy as np
4
5 # Configurar cross-validation temporal
6 # n_splits=5 crea 5 folds
7 # test_size: tamaño de cada conjunto de prueba (opcional)
8 # gap: gap entre train y test (opcional, para evitar data
  leakage)
9 tscv = TimeSeriesSplit(n_splits=5)
10
```

```

11 mae_scores = []
12 rmse_scores = []
13
14 for fold, (train_index, test_index) in enumerate(tscv.split(
    ts)):
15     train = ts.iloc[train_index]
16     test = ts.iloc[test_index]
17
18     print(f"Fold {fold+1}: Train size={len(train)}, Test size
    ={len(test)}")
19
20     # Entrenar modelo (ejemplo con ARIMA)
21     modelo = ARIMA(train, order=(1,1,1))
22     fit = modelo.fit()
23
24     # Predecir
25     predicciones = fit.forecast(steps=len(test))
26
27     # Calcular metricas
28     mae = mean_absolute_error(test, predicciones)
29     rmse = np.sqrt(mean_squared_error(test, predicciones))
30
31     mae_scores.append(mae)
32     rmse_scores.append(rmse)
33     print(f"    MAE: {mae:.2f}, RMSE: {rmse:.2f}")
34
35 print(f"\nMAE promedio: {np.mean(mae_scores):.2f} (+/- {np.
    std(mae_scores):.2f})")
36 print(f"RMSE promedio: {np.mean(rmse_scores):.2f} (+/- {np.
    std(rmse_scores):.2f})")

```

6.3.4. Implementación Manual de Expanding Window

Si queremos usar expanding window, podemos implementarlo manualmente:

```

1 # Expanding window: el train crece en cada iteracion
2 train_size_initial = int(len(ts) * 0.3) # Empezar con 30% de
    los datos
3 n_folds = 5
4
5 mae_scores = []
6 rmse_scores = []
7
8 for i in range(n_folds):
9     # El train crece en cada iteracion

```

```

10     train_end = train_size_initial + i * (len(ts) -
train_size_initial) // n_folds
11     test_start = train_end
12     test_end = train_end + (len(ts) - train_size_initial) //
n_folds
13
14     train = ts[:train_end]
15     test = ts[test_start:test_end]
16
17     # Entrenar y evaluar modelo
18     modelo = ARIMA(train, order=(1,1,1))
19     fit = modelo.fit()
20     predicciones = fit.forecast(steps=len(test))
21
22     mae = mean_absolute_error(test, predicciones)
23     rmse = np.sqrt(mean_squared_error(test, predicciones))
24
25     mae_scores.append(mae)
26     rmse_scores.append(rmse)

```

6.4. Comparación de Modelos

```

1 import pandas as pd
2 from sklearn.metrics import mean_absolute_error,
mean_squared_error
3 import numpy as np
4
5 # Dividir datos
6 train = ts[:18]
7 test = ts[18:]
8
9 # Modelo 1: Media Movil Simple
10 sma_pred = train.rolling(window=3).mean().iloc[-1]
11 sma_pred = [sma_pred] * len(test) # Prediccion constante
12
13 # Modelo 2: Holt-Winters
14 from statsmodels.tsa.holtwinters import ExponentialSmoothing
15 hw_model = ExponentialSmoothing(train, trend='add', seasonal=
'add',
16                                     seasonal_periods=12).fit()
17 hw_pred = hw_model.forecast(steps=len(test))
18
19 # Modelo 3: ARIMA
20 arima_model = ARIMA(train, order=(1,1,1)).fit()

```

```

21 arima_pred = arima_model.forecast(steps=len(test))
22
23 # Calcular metricas
24 resultados = {
25     'Modelo': ['Media Movel', 'Holt-Winters', 'ARIMA'],
26     'MAE': [
27         mean_absolute_error(test, sma_pred),
28         mean_absolute_error(test, hw_pred),
29         mean_absolute_error(test, arima_pred)
30     ],
31     'RMSE': [
32         np.sqrt(mean_squared_error(test, sma_pred)),
33         np.sqrt(mean_squared_error(test, hw_pred)),
34         np.sqrt(mean_squared_error(test, arima_pred))
35     ]
36 }
37
38 df_resultados = pd.DataFrame(resultados)
39 print(df_resultados)

```

6.5. Intervalos de Confianza

Los modelos estadísticos como ARIMA y Holt-Winters proporcionan intervalos de confianza:

```

1 # ARIMA con intervalos de confianza
2 predicciones_arima = fit.get_forecast(steps=6)
3 pred_mean = predicciones_arima.predicted_mean
4 pred_ci = predicciones_arima.conf_int(alpha=0.05) # 95%
5         confianza
6
7 plt.figure(figsize=(12, 6))
8 plt.plot(ts, label='Datos Historicos')
9 plt.plot(pred_mean, label='Prediccion', color='red')
10 plt.fill_between(pred_ci.index,
11                 pred_ci.iloc[:, 0],
12                 pred_ci.iloc[:, 1],
13                 color='red', alpha=0.3,
14                 label='Intervalo 95%')
15 plt.legend()
16 plt.show()

```